

ISIT312/ISIT912 Big Data Management

Spring 2025

OLAP Operations in HQL

In this practice, you will learn how to use Hive HQL extensions for Data Warehousing. A laboratory includes application of **SELECT** statement with **GROUP BY** clause, advanced features of **GROUP BY** clause (**ROLLUP**, **CUBE**, **GROUPING SETS**), windowing and analytics functions.

Warning: To copy the fragments of code from this document to your working Terminal use evince pdf viewer.

Software Installation and Setup

If you use a system installed in 3.230 i.e. a laboratory room for ISIT312/912, then you do not need to install VirtualBox.

If you use your system (laptop) then start from installation of the latest version of VirtualBox and the extension pack (optional). The distribution packages are available at <https://www.virtualbox.org/wiki/Downloads> for Windows, MacOS (Intel processor only) and Linux.

If you use your system (laptop) then you must copy a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Hive4.0.ova` file located at `C:\Program Files (x86)\ISIT312` and `ISIT912` (if you cannot find the file in this location please ask a tutor about correct location) folder to your system. You can also download the file from Moodle.

Next, start VirtualBox and import the file. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

If you use a system installed in 3.230 then start VirtualBox and import a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Hive4.0.ova` file located at `C:\Program Files (x86)\ISIT312` and `ISIT912` folder into VirtualBox. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

After the import is completed, a virtual machine named `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Hive4.0.1` appears in the VirtualBox.

Laboratory Instructions

Start a virtual machine `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Hive4.0.1`

Note, that both the account name and the password are: `bigdata`

(0) Start Shell

After you log on the Ubuntu system with a user name `bigdata` and password `bigdata`, start a Terminal window with `Ctrl + Alt + T` or use the fifth from top icon located at a vertical stripe (sidebar) on the left-hand side of the screen.

The documents `LinuxCommandLineCheatSheet.pdf` and `Efficient-Linux-at-the-Command-Line-ch4.pdf` available through `Readings` link in `Resources` section in `linux` folder on Moodle contain more information on how to use Linux Shell in Terminal window.

You can use the Terminal window to interact with Hadoop. A simple hint that may make your life much easier is to use "Up" and "Down" keyboard keys to navigate through the commands already processed in Terminal window.

Now you can interact with Hadoop.

(1) Hadoop Initialisation

Now you can start Hadoop. Process a shell script `start-all.sh` in Terminal window in the following way.

```
$HADOOP_HOME/sbin/start-all.sh
```

The system will display few warnings and the following messages:

```
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bigdata-VirtualBox]
Starting resourcemanager
Starting nodemanagers
```

After you ignore the warning messages you can view the running processes with a command:

```
jps
```

The following results should be returned (note that the process numbers may be different).

```
819 NodeManager
2501 SecondaryNameNode
2310 DataNode
4391 Jps
2187 NameNode
2700 ResourceManager
```

Laboratory Instructions

(2) How to start command line interface `beeline` ?

To start a command line interface `beeline` process the following command in Terminal window.

```
beeline
```

You should get few log messages, a beeline welcome line

```
Beeline version 4.0.1 by Apache Hive
```

and a prompt.

```
beeline>
```

To connect beeline to Hive, process the following statement in front of beeline> prompt.

```
!connect jdbc:hive2://
```

You should get the following messages.

```
Connecting to jdbc:hive2://  
Enter username for jdbc:hive2://:
```

Press Enter when prompted about user name. You should get a message.

```
Enter password for jdbc:hive2://:
```

Press Enter when prompted about password.

The system should reply with the following messages

```
Hive Session ID = b637dfc8-7b04-4da7-996d-87dc279404c0  
Connected to: Apache Hive (version 4.0.1)  
Driver: Hive JDBC (version 4.0.1)  
Transaction isolation: TRANSACTION_REPEATABLE_READ
```

and with a prompt.

```
0: jdbc:hive2://>
```

To find what databases are available process the following statement.

```
show databases;
```

At the moment only default database is available. We shall create new databases in the future.

To find what tables have been created so far process the following statement.

```
show tables;
```

In the following steps, we use beeline to interact with Hive.

(3) How to create and how to load data into an *internal* table ?

We shall use the default database for Hive table created, loaded, and used in this laboratory exercise.

Create the following internal table to store information about items.

```
create table ORDERS(  
  part      char(7),  
  customer  varchar(30),  
  quantity  decimal(8),  
  oyear     decimal(4),  
  omonth    decimal(2),  
  oday      decimal(2) )  
  row format delimited fields terminated by ','  
  stored as textfile;
```

The table represents a three-dimensional data cube. A fact entity `orders` is described by a measure quantity. The dimensions include `part`, `customer`, and obviously time (`oyear`, `omonth`, `oday`) dimension. There is a hierarchy over time dimension where years consist of months and months consist of days.

Next create a text file `orders.txt` with sample data given below and save the file in a folder where you plan to keep HQL scripts from this lab. It is assumed that you have already started `beeline` from this folder. If you started `beeline` from a different folder then quit it, move to a folder where a file `orders.txt` is located and restart `beeline`.

```
bolt,James,200,2016,01,01  
bolt,Peter,100,2017,01,30  
bolt,Bob,300,2018,05,23  
screw,James,20,2017,05,11  
screw,Alice,55,2018,01,01  
nut,Alice,23,2018,03,16  
washer,James,45,2016,04,24  
washer,Peter,100,2016,05,12  
bolt,James,200,2018,01,05  
bolt,Peter,100,2018,01,05
```

To load data into a table `orders` process the following load statement.

```
load data local inpath '.../orders.txt' into table orders;
```

"..." is your path to a file `item.txt`. Verify the contents of a table `orders` with a simple

```
select *  
from orders;
```

(4) How to perform a simple aggregation with `group by` and `having` clauses ?

We start from implementation of a query that *finds the total number of orders per each part*, i.e. we perform aggregation along a dimension `part`. Process the following `select` statement.

```
select part, count(*)  
from orders  
group by part;
```

Next, we *find the total number of parts summarized per each part*. It is another aggregation along a dimension `part`. Process the following `select` statement.

```
select part, sum(quantity)
from orders
group by part;
```

Next, we *find the total number of orders per customer* and we list only the customers who submitted more than one order. Process the following `select` statement.

```
select customer, count(*)
from orders
group by customer
having count(*) > 1;
```

Now, assume that we would like to *find in one query the total number of orders per each part, per each customer and per both part and customer*. Process the following `select` statement.

```
select part, NULL, count(*)
from orders
group by part
union
select NULL, customer, count(*)
from orders
group by customer
union
select part, customer, count(*)
from orders
group by part, customer;
```

Implementation if a query given above is terribly inefficient. It is a perfect example of a very bad SQL. To perform the aggregations a relational table `orders` is sequentially scanned three times. The same aggregations can be computed in single scan through a relational table `orders`. A problem is, how to syntactically express a query that performs three aggregations and a relational table `orders` is used in `from` clause just one time.

(5) How to perform aggregations with `rollup` operator ?

Assume that we would like to perform aggregation over two dimensions, then over one of the two dimensions used earlier and then aggregation over all rows in a table. For example, *find the total number of parts ordered and summarized per part and per customer, then per part and then total number of all parts ordered*. A sample solution given below is an example very inefficient implementation of the aggregation.

```
select part, customer, sum(quantity)
from orders
group by part, customer
union
select part, null, sum(quantity)
from orders
group by part
union
select null, null, sum(quantity)
from orders;
```

Implementation above is very inefficient because a relational table `orders` is sequentially read three times. While, all summations over different dimensions can be computed in a single pass through a relational table `orders`. A The same query can be implemented as a single `select` statement with `rollup` operator in the following way.

```
select part, customer, sum(quantity)
from orders
group by part, customer with rollup;
```

In the next example we use `rollup` operator to implement a query that *finds the total number of parts ordered and summarized per year and month, per year, and the total number of parts ordered.*

```
select oyear, omonth, sum(quantity)
from orders
group by oyear,omonth with rollup;
```

(6) How to perform aggregation with `cube` operator ?

Assume that we would like to *find an average number of parts ordered and summarized per part, per customer, both per part and customer and an average number of parts per order.* An implementation that uses `cube` operator is given below.

```
select part, customer, avg(quantity)
from orders
group by part, customer with cube;
```

It is possible to verify some of the results with the following queries.

```
select avg(quantity)
from orders;
```

and

```
select part, avg(quantity)
from orders
where part='bolt'
group by part;
```

(7) How to perform aggregations with `grouping sets` operator ?

Assume that we would like to *find the total number of orders per part and per customer and both per year and customer.* A sample implementation of the query with `grouping sets` operator is the following.

```
select part, customer, oyear, count(*)
from ORDERS
group by part, customer, oyear
grouping sets ((part),(customer), (oyear,customer));
```

In another example we *find the total number of parts ordered and summarized per year, month, day, per year and month, and per year, and the total number of parts ordered.*

```
select oyear, omonth, oday, sum(quantity)
from orders
group by oyear,omonth,oday grouping
sets((oyear,omonth,oday),(oyear,omonth),(oyear),());
```

Note that a query given above returns the same results as the following query with `rollup` operator.

```
select oyear, omonth, oday, sum(quantity)
from orders
group by oyear,omonth,oday with rollup;
```

Do you know how to implement a query with `cube` operator as a query with `grouping sets` operator ?

(8) How to perform window based aggregations ?

It is possible to use `group by` clause of `select` statement to find the total number of ordered parts summarized per each part.

```
select part, sum(quantity)
from orders
group by part;
```

It is possible to get the similar result as from a query with `group by` clause with so called *windowing*.

```
select part, SUM(quantity) over (partition by part)
from orders;
```

bolt	900.00
bolt	900.00
bolt	900.00
bolt	900.00
bolt	900.00
nut	23.00
screw	75.00
screw	75.00
washer	145.00
washer	145.00

To get the same results we have to use `distinct` keyword.

```
select distinct part, SUM(quantity) over (partition by part)
from orders;
```

bolt	900.00
nut	23.00
screw	75.00
washer	145.00

Next, we use windowing to implement a query that *finds for each part, for each customer, and for each quantity ordered by customer the largest total number of parts ordered and aggregated per part*.

```
select part, customer, quantity, MAX(quantity) over (partition by
part)
from orders;
```

bolt	Peter	100.00	300.00
bolt	James	200.00	300.00
bolt	Bob	300.00	300.00
bolt	Peter	100.00	300.00
bolt	James	200.00	300.00
nut	Alice	23.00	23.00
screw	Alice	55.00	55.00
screw	James	20.00	55.00
washer	Peter	100.00	100.00
washer	James	45.00	100.00

A table `orders` is partitioned (grouped by) the values in column `part` and for each part the largest quantity is found and added to each output row that consists of `part`, `customer` and `quantity`.

It is possible to use more than one aggregation. For example, we can extend a query above with the *summarization of the quantities per each part* in the following way.

```
select part, customer, quantity,
       MAX(quantity) over (partition by part),
       SUM(quantity) over (partition by part)
from orders;
```

(9) How to perform window aggregations and window ordering ?

We start from a query that *finds for each part an quantity ordered and the total number of all parts ordered*. Such query can be implemented in the following way:

```
select part, quantity, SUM(quantity) over (partition by part)
from orders;
```

The results are the following.

bolt	100.00	900.00
bolt	200.00	900.00
bolt	300.00	900.00
bolt	100.00	900.00
bolt	200.00	900.00
nut	23.00	23.00
screw	55.00	75.00
screw	20.00	75.00
washer	100.00	145.00
washer	45.00	145.00

Now, we add a clause `order by` to *windowing*. Process the following statement.

```
select part, quantity,
       SUM(quantity) over (partition by part order by quantity)
from orders;
```

bolt	100.00	200.00	<-- 100+100
bolt	100.00	200.00	<-- 100+100
bolt	200.00	600.00	<-- 200+200
bolt	200.00	600.00	<-- 200+200
bolt	300.00	900.00	
nut	23.00	23.00	
screw	20.00	20.00	
screw	55.00	75.00	
washer	45.00	45.00	
washer	100.00	145.00	

Addition of `order by` clause computes the increasing results of summarization over the quantities and sorts the rows in each partition by the summarized quantity. If two rows have the same values of `order by quantity` then the rows are treated as one row with summarized quantity. For example the first two rows have the same values of `order by quantity` and because of that a value of `SUM(quantity) = 100+100`. The same applies to the next two rows. If two or more rows have the same values of part and quantity then summarization is performed in one step over all such rows. This problem (if it is really a problem ?) can be solved with more selective `order by` key. For example, the rows in each window can be ordered by `quantity, oyear, omonth, and oday`.

```
select part, quantity,
       SUM(quantity) over (partition by part
                          order by quantity, oyear, omonth, oday)
from orders;
```

In this case the rows in each window are ordered by `quantity, oyear, omonth, oday` and summarization is performed in a row-by-row mode. The sample results are given below.

bolt	100.00	100.00	SUM(100)
bolt	100.00	200.00	SUM(100+100)
bolt	200.00	400.00	SUM(100+100+200)
bolt	200.00	600.00	SUM(100+100+200+200)
bolt	300.00	900.00	SUM(100+100+200+200+300)
nut	23.00	23.00	SUM(23)
screw	20.00	20.00	SUM(20)
screw	55.00	75.00	SUM(20+55)
washer	45.00	45.00	SUM(45)
washer	100.00	145.00	SUM(45+100)

To find how the ordered quantities of ordered parts changed year by year process the following select statement.

```
select part, quantity, oyear,
       SUM(quantity) over (partition by part order by oyear)
from orders;
```

bolt	200.00	2016	200.00	SUM(200)
bolt	100.00	2017	300.00	SUM(200+100)
bolt	100.00	2018	900.00	SUM(200+100+100+200+300)

bolt	200.00	2018	900.00	SUM(200+100+100+200+300)
bolt	300.00	2018	900.00	SUM(200+100+100+200+300)
nut	23.00	2018	23.00	SUM(23)
screw	20.00	2017	20.00	SUM(20)
screw	55.00	2018	75.00	SUM(20+55)
washer	100.00	2016	145.00	SUM(100+45)
washer	45.00	2016	145.00	SUM(100+45)

Now, we change an aggregation function to AVG.

```
select part, quantity, oyear,
       AVG(quantity) over(partition by part order by oyear)
from orders;
```

The statement finds so called *walking average*.

bolt	200.00	2016	200.000000	AVG(200)
bolt	100.00	2017	150.000000	AVG(200+100)
bolt	100.00	2018	180.000000	AVG(200+100+100+200+300)
bolt	200.00	2018	180.000000	AVG(200+100+100+200+300)
bolt	300.00	2018	180.000000	AVG(200+100+100+200+300)
nut	23.00	2018	23.000000	AVG(23)
screw	20.00	2017	20.000000	AVG(20)
screw	55.00	2018	37.500000	AVG(20+55)
washer	100.00	2016	72.500000	AVG(100+45)
washer	45.00	2016	72.500000	AVG(100+45)

(10) How to perform window aggregations and window framing ?

Next, implement a query that for each part and quantity *finds an average of quantity ordered by year, month and day*. Process the following statement.

```
select part, quantity,
       AVG(quantity) over (partition by part
                          order by oyear, omonth, oday)
from orders;
```

bolt	200.00	200.000000	AVG(200)
bolt	100.00	150.000000	AVG(200+100)
bolt	100.00	150.000000	AVG(200+100+100+200)
bolt	200.00	150.000000	AVG(200+100+100+200)
bolt	300.00	180.000000	AVG(200+100+100+200+300)
nut	23.00	23.000000	AVG(23)
screw	20.00	20.000000	AVG(20)
screw	55.00	37.500000	AVG(20+55)
washer	45.00	45.000000	AVG(45)
washer	100.00	72.500000	AVG(45+100)

Processing of aggregation (average) is performed over an expanding frame. At the beginning a *frame* includes the first row, next the first row and the second row, next the first 3 rows, etc.

It is possible to create a fixed size frame smaller than a window. Process the following statement.

```
select part, quantity,
```

```

        AVG(quantity) over (partition by part
                           order by oyear, omonth, oday
                           rows 1 preceding)
from orders;

```

The statement finds for each part and quantity an average quantity of the current and previous one quantity when the quantities are sorted in time (please see the next page).

bolt	200.00	200.000000	AVG (200)
bolt	100.00	150.000000	AVG (200+100)
bolt	100.00	100.000000	AVG (100+100)
bolt	200.00	150.000000	AVG (100+200)
bolt	300.00	250.000000	AVG (200+300)
nut	23.00	23.000000	AVG (23)
screw	20.00	20.000000	AVG (20)
screw	55.00	37.500000	AVG (20+55)
washer	45.00	45.000000	AVG (45)
washer	100.00	72.500000	AVG (45+100)

Also, note that processing of the following statement

```

select part, quantity,
        AVG(quantity) over (partition by part
                           order by oyear, omonth, oday
                           rows unbounded preceding)
from orders;

```

returns the same results as processing of

```

select part, quantity,
        AVG(quantity) over (partition by part
                           order by oyear, omonth, oday
                           rows between unbounded preceding and current row)
from orders;

```

The options of *window framing* are the following.

```

(ROWS RANGE) BETWEEN (UNBOUNDED [num]) PRECEDING AND ([num] PRECEDING
CURRENT ROW (UNBOUNDED [num]) FOLLOWING)

```

For example:

```

ROWS BETWEEN 3 PRECEDING AND CURRENT ROW,
ROWS BETWEEN UNBOUNDED PRECEDING AND 2 FOLLOWING

```

```

(ROWS RANGE) BETWEEN CURRENT ROW AND (CURRENT ROW (UNBOUNDED [num])
FOLLOWING)

```

For example:

```

ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING

```

```

(ROWS RANGE) BETWEEN [num] FOLLOWING AND (UNBOUNDED [num]) FOLLOWING

```

For example:

```

ROWS BETWEEN 2 FOLLOWING AND UNBOUNDED FOLLOWING

```

(11) How to use window clause ?

It is possible to simplify syntax a bit with window clause (definition). Implement the following query.

```
select part, SUM(quantity) over w
from orders
window w as (partition by part);
```

(12) How to use LEAD and LAG functions ?

LEAD and LAG functions allow to access the next and previous values in a column, respectively. For example, we would like to *find the current and the next quantity for each part ordered by year, month, day*. Process the following statement.

```
select part, quantity,
       LEAD(quantity) over (partition by part
                           order by oyear, omonth, oday)
from orders;
```

bolt	200.00	100.00
bolt	100.00	100.00
bolt	100.00	200.00
bolt	200.00	300.00
bolt	300.00	
nut	23.00	
screw	20.00	55.00
screw	55.00	
washer	45.00	100.00
washer	100.00	

Next, we would like to *find the current and the previous quantity for each part ordered by year, month, day*. Process the following statement.

```
select part, quantity,
       LAG(quantity) over (partition by part
                          order by oyear, omonth, oday)
from orders;
```

bolt	200.00	
bolt	100.00	200.00
bolt	100.00	100.00
bolt	200.00	100.00
bolt	300.00	200.00
nut	23.00	
screw	20.00	
screw	55.00	20.00
washer	45.00	
washer	100.00	45.00

Next we subtract the previous row value from the current row value. Process the following statement.

```

select part, quantity,
       quantity - LAG(quantity) over(partition by part
                                     order by oyear, omonth, oday)
from orders;

```

bolt	200.00	
bolt	100.00	-100.00
bolt	100.00	0.00
bolt	200.00	100.00
bolt	300.00	100.00
nut	23.00	
screw	20.00	
screw	55.00	35.00
washer	45.00	
washer	100.00	55.00

Empty places (NULLs) can be eliminated with a parameter 0 in LAG function. Process the following statement.

```

select part, quantity,
       quantity+lag(quantity,1,0) over(partition by part
                                     order by oyear, omonth, oday)
from orders;

```

bolt	200.00	200.00
bolt	100.00	300.00
bolt	100.00	200.00
bolt	200.00	300.00
bolt	300.00	500.00
nut	23.00	23.00
screw	20.00	20.00
screw	55.00	75.00
washer	45.00	45.00
washer	100.00	145.00

(13) How to use analytic functions ?

Finally, we implement windowing with the analytic functions RANK(), DENSE_RANK(), CUM_DIST(),

A function RANK() assigns a rank to row such that the rows with the same value of quantity are ranked with the same number and rank is increased by the total number of rows with the same value. Process the following statement.

```

select part, quantity,
       RANK() over (partition by part order by quantity)
from orders;

```

bolt	100.00	1	RANK=1
bolt	100.00	1	RANK=1
bolt	200.00	3	RANK=1+2

bolt	200.00	3	RANK=1+2
bolt	300.00	5	RANK=3+2
nut	23.00	1	
screw	20.00	1	
screw	55.00	2	
washer	45.00	1	
washer	100.00	2	

A function `DENSE_RANK()` assigns a rank to row such that the rows with the same value of quantity are ranked with the same number and rank is increased by 1 for each group of rows with the same value of quantity. Process the following statement.

```
select part, quantity,
       DENSE_RANK() over (partition by part
                          order by quantity)
from orders;
```

bolt	100.00	1
bolt	100.00	1
bolt	200.00	2
bolt	200.00	2
bolt	300.00	3
nut	23.00	1
screw	20.00	1
screw	55.00	2
washer	45.00	1
washer	100.00	2

A function `CUME_DIST()` computes the relative position of a specified value in a group of values. For a given row *r*, the `CUME_DIST()` the number of rows with values lower than or equal to the value of *r*, divided by the number of rows being evaluated, i.e. entire window. Process the following statement.

```
select part, quantity,
       CUME_DIST() over (partition by part
                          order by quantity)
from orders;
```

bolt	100.00	0.4	2 rows/5
bolt	100.00	0.4	2 rows/5
bolt	200.00	0.8	4 rows/5
bolt	200.00	0.8	4 rows/5
bolt	300.00	1.0	5 rows/5
nut	23.00	1.0	
screw	20.00	0.5	
screw	55.00	1.0	
washer	45.00	0.5	
washer	100.00	1.0	

A function `PERCENT_RANK()` is similar to a function `CUME_DIST()`. For a row r , `PERCENT_RANK()` calculates the rank of r minus 1, divided by the number of rows being evaluated -1, i.e. entire window-1. Process the following statement.

```
select part, quantity,
       PERCENT_RANK() over (partition by part
                           order by quantity)
from orders;
```

bolt	100.00	0.0
bolt	100.00	0.0
bolt	200.00	0.5
bolt	200.00	0.5
bolt	300.00	1.0
nut	23.00	0.0
screw	20.00	0.0
screw	55.00	1.0
washer	45.00	0.0
washer	100.00	1.0

A function `NTILE(k)` divides a window into a number of buckets indicated by k and assigns the appropriate bucket number to each row. The buckets are numbered from 1 to k . Process the following statement.

```
select part, quantity,
       NTILE(2) over (partition by part
                     order by quantity)
from orders;
```

bolt	100.00	1
bolt	100.00	1
bolt	200.00	1
bolt	200.00	2
bolt	300.00	2
nut	23.00	1
screw	20.00	1
screw	55.00	2
washer	45.00	1
washer	100.00	2

```
select part, quantity,
       NTILE(5) over (partition by part
                     order by quantity)
from orders;
```

bolt	100.00	1
bolt	100.00	2
bolt	200.00	3
bolt	200.00	4
bolt	300.00	5
nut	23.00	1
screw	20.00	1
screw	55.00	2

washer	45.00	1
washer	100.00	2

Finally, a function ROW_NUMBER does not need any explanations. Process the following statement.

```
select part, quantity,
       ROW_NUMBER() over (partition by part
                          order by quantity)
from orders;
```

bolt	100.00	1
bolt	100.00	2
bolt	200.00	3
bolt	200.00	4
bolt	300.00	5
nut	23.00	1
screw	20.00	1
screw	55.00	2
washer	45.00	1
washer	100.00	2
